2026-7-15 Linux进程--进程控制

- 1. 进程控制
  - 1.1 进程创建
    - 1.1.1 fork() 为什么能返回两次
    - 1.1.2 怎样通过返回值区分父子进程
    - 1.1.3 写时拷贝与进程独立性
    - 1.1.4 常见用法与失败原因
  - 1.2 进程终止
    - 1.2.1 进程的三种结束结果
    - 1.2.2 退出码与库函数错误码
    - 1.2.3 return、exit() 与 _exit()
    - 1.2.4 退出不等于所有信息立即被回收
  - 1.3 进程等待
    - 1.3.1 为什么父进程必须回收子进程
    - 1.3.2 wait() 与 waitpid()
    - 1.3.3 解码 status
    - 1.3.4 正常等待与异常退出
    - 1.3.5 等待失败与孤儿时间
    - 1.3.6 使用宏提取退出信息
    - 1.3.7 阻塞等待、非阻塞轮询与多进程
  - 1.4 进程程序替换
    - 1.4.1 exec 替换什么，不替换什么
    - 1.4.2 exec 函数族的命名规则
    - 1.4.3 fork() 与 exec() 配合
    - 1.4.4 从函数调用理解进程控制
  - 1.5 用 shell 串起进程控制
    - 1.5.1 shell 的核心循环
    - 1.5.2 为什么有些命令必须由 shell 自己执行
    - 1.5.3 环境变量、命令行参数与退出状态
  - 1.6 小结

1. 进程控制

进程控制研究的是一条完整的生命周期：进程怎样被创建，怎样退出，退出后由谁回收，又怎样把当前程序换成另一个程序。把这些操作串起来，就得到一个命令行解释器最基本的运行模型。

可以把主线先概括为：

> `fork()` 创建执行流 → 子进程 `exit()` 退出 → 父进程 `wait()` 回收 → `exec()` 替换程序 → shell 循环重复整个过程。

当前笔记只讨论进程控制。地址空间本身的结构和页表细节，参见 2026-7-15 Linux进程--程序地址空间。

1.1 进程创建

1.1.1 `fork()` 为什么能返回两次

Linux 可以使用 `fork()` 从已有进程创建一个新进程：

```
#include <unistd.h>

pid_t fork(void);
```

调用 `fork()` 的进程是父进程，新产生的进程是子进程。调用发生后，内核大致完成以下工作：

1. 为子进程分配新的 `task_struct` 等内核数据结构；
2. 复制父进程中适合由子进程继承的管理信息；
3. 为子进程建立独立的执行上下文；
4. 把子进程加入可被调度器管理的进程集合；
5. 父进程和子进程分别从 `fork()` 的返回处继续执行。

因此，不能说“一个函数执行完返回了两次”。更准确地说，`fork()` 之后系统中已经有了两个执行流，只是两个执行流都从同一个程序位置继续运行。

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("before fork: pid = %d\n", getpid());
    fflush(stdout);

    pid_t id = fork();
```

```
    if (id < 0) {
        perror("fork");
        return 1;
    } else if (id == 0) {
        printf("child: pid = %d, parent = %d\n",
               getpid(), getppid());
    } else {
        printf("parent: pid = %d, child = %d\n",
               getpid(), id);
    }

    return 0;
}
```

程序不会出现两次 `before fork`，因为父进程执行到 `fork()` 之前，子进程还不存在。`fork()` 创建子进程之后，父子进程都能继续执行后面的分支。

1.1.2 怎样通过返回值区分父子进程

`fork()` 在父进程中返回子进程的 PID，在子进程中返回 `0`，创建失败时返回 `-1`：

位置	`fork()` 返回值	判断方法
父进程	新创建子进程的 PID，通常大于 `0`	`id > 0`
子进程	`0`	`id == 0`
创建失败	`-1`	`id < 0`

子进程返回 `0`，不是因为它没有身份，而是为了给程序提供一个简单可靠的子进程分支条件。子进程仍可通过 `getpid()` 获取自己的 PID，通过 `getppid()` 获取父进程 PID。

`fork()` 后父子进程谁先得到 CPU 并不固定，由调度器决定。不能根据源码中父子分支的书写顺序推断实际运行顺序。

1.1.3 写时拷贝与进程独立性

创建子进程时，没必要立即把父进程的全部物理数据复制一份。父子进程可以先共享只读的代码页和数据页，再由页表分别记录映射关系。

当任意一方尝试修改共享数据时，内核才为该进程复制对应页面。这就是写时拷贝，也就是COW。

写时拷贝保证了：

- 子进程初始看到的变量内容与父进程相近；
- 父子进程具有各自独立的虚拟地址空间；
- 未写入的物理页可以共享，减少创建成本；
- 一旦写入，修改只影响执行写入的进程；
- 进程之间不会因为普通的全局变量修改而直接互相干扰。

`fork()` 并不是只复制代码。子进程通常还会继承当前工作目录、打开的文件描述符、环境变量和大量进程属性；这些资源如何继承，要结合具体对象判断。

1.1.4 常见用法与失败原因

`fork()` 的常见用法有两类：

1. 父进程继续处理总任务，子进程分别处理某个请求，例如服务器为每个连接创建子进程；
2. 子进程先调用 `fork()` 得到新的执行流，随后再调用 `exec()` 执行另一个程序，这是shell 运行外部命令的典型方式。

`fork()` 可能因为系统资源不足而失败，常见原因包括：

- 系统或用户的进程数量达到限制；
- 内核无法分配新的任务结构；
- 系统没有足够的资源完成地址空间和页表复制。

实际代码必须检查返回值。创建失败后直接执行父进程或子进程分支，会把错误掩盖成普通控制流。

1.2 进程终止

1.2.1 进程的三种结束结果

一个进程运行结束，通常要回答两个不同问题：

1. 它是正常执行结束，还是被信号异常终止？
2. 如果正常结束，它给出的退出码是多少？

从代码观察，可以分成三种情况：

情况	信号编号	退出码	含义
代码运行完成，结果正确	`0`	通常为 `0`	正常成功结束
代码运行完成，结果错误	`0`	非 `0`	正常结束，但业务结果失败
代码未运行完，被信号终止	非 `0`	无意义	异常结束

例如，子进程发生空指针解引用时会被 `SIGSEGV` 终止。此时程序不一定执行到 `return` 或 `exit()`，因此不能再用普通退出码解释失败原因。

进程结束时，内核会把与终止有关的信息保留在任务结构中，供父进程以后读取。父进程等待时拿到的 `status` 同时携带“是否正常结束”“退出码是多少”“被哪个信号终止”等信息。

1.2.2 退出码与库函数错误码

最简单的退出码检查方式是：

```
./program
echo $?
```

`$?` 展示上一条命令的退出状态。退出码 `0` 一般表示成功，非零值常用来表示不同类型的失败。具体退出码和含义应由程序设计约定，不能只凭数字猜测业务原因。

可以定义自己的退出码：

```
#define SUCCESS 0
#define USAGE_ERROR 1
#define OPEN_ERROR 2
```

这类宏属于程序自己约定的语义。例如：

```c
int main(int argc, char *argv[])
{
    if (argc != 2) {
        return USAGE_ERROR;
    }

    return SUCCESS;
}
```

进程退出码和库函数错误码是两套概念：

- 系统调用或库函数失败时，常通过 `errno` 报告具体原因；
- 进程结束时通过退出码告诉父进程整体执行结果；
- `strerror(errno)` 可以把 `errno` 转换成可读的错误描述。

Linux 中真正提供给父进程的退出码通常只保留低 8 位，所以有效范围是 `0` 到 `255`。例如把一个更大的值作为退出码时，父进程看到的会是截断或取模后的结果。

1.2.3 `return`、`exit()` 与 `_exit()`

进程常见的正常退出方式有：

1. 从 `main()` 执行 `return n`；
2. 在任意函数中调用 `exit(n)`；
3. 在任意函数中调用 `_exit(n)`。

从 `main()` 返回时，C 运行时会把返回值作为退出码，并完成相应清理，效果接近调用 `exit()`。

普通函数中的 `return` 只结束当前函数，不会结束进程；普通函数中的 `exit()` 或 `_exit()` 会结束整个进程。

`exit()` 与 `_exit()` 的关键区别是：

- `exit()` 先执行 `atexit()` 注册的清理函数；
- `exit()` 刷新并关闭 C 标准库缓冲区；
- `_exit()` 直接结束进程，不执行用户态的清理和缓冲区刷新。

因此，`exit()` 和 `_exit()` 不能简单说成完全相同。通常需要正常清理时使用 `return` 或 `exit()`；在已经发生严重错误、或 `fork()` 后不希望重复刷新父进程继承的缓冲区时，可能需要 `_exit()`。

1.2.4 退出不等于所有信息立即被回收

进程结束时，内核会回收它的用户空间代码、数据和大部分资源，但不会立刻把所有内核管理信息都删掉。

子进程已经退出、父进程还没有读取其退出状态的阶段，子进程处于僵尸状态。僵尸进程的主体已经死亡，继续保留少量进程信息，以便父进程获得退出原因和退出码。

如果父进程先退出，子进程通常会由 `init` 或 `systemd` 等收养。孤儿进程仍可继续运行；只有在它们退出以后，也需要有人负责回收。

1.3 进程等待

1.3.1 为什么父进程必须回收子进程

父进程等待子进程，主要有三个目的：

1. 回收子进程遗留的内核资源，避免僵尸进程长期占用进程表；
2. 获得子进程的退出状态，判断任务是否成功；
3. 控制并发关系，让父进程知道某个后台工作已经结束。

僵尸进程已经结束，`kill -9` 不能把僵尸“再杀死一次”。正确解决办法是让父进程或新的收养者调用等待函数读取状态，完成回收。

1.3.2 `wait()` 与 `waitpid()`

`wait()` 等待任意一个子进程结束：

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
```

成功时返回被回收子进程的 PID；失败时返回 `-1`。如果不关心退出状态，可以传入 `NULL`。

`waitpid()` 可以指定等待目标：

```
pid_t waitpid(pid_t pid, int *status, int options);
```

常用 `pid` 取值如下：

`pid`	含义
`pid > 0`	等待 PID 等于 `pid` 的指定子进程
`pid == -1`	等待任意一个子进程，接近 `wait()` 的行为
`pid == 0`	等待同一进程组中的任意子进程
`pid < -1`	等待指定进程组中的子进程

常见 `options` 取值：

- `0`：阻塞等待，直到目标子进程结束或出错；
- `WNOHANG`：非阻塞检查，目标还未结束时立即返回 `0`。

`waitpid()` 的成功返回值是被回收的子进程 PID；使用 `WNOHANG` 且暂时没有子进程退出时返回 `0`；失败时返回 `-1` 并设置 `errno`。

1.3.3 解码 `status`

`status` 不能直接当成普通整数解释。父进程只使用它的低 16 位：

比特位	含义
第 0-6 位	导致终止的信号编号；正常退出时为 `0`
第 7 位	是否产生 core dump
第 8-15 位	正常退出时的退出码
更高位	当前场景不使用

因此，手写位运算时常会看到：

```
int signal_number = status & 0x7F;
int exit_code = (status >> 8) & 0xFF;
```

手动位运算适合理解结构，实际代码更应优先使用标准宏：

```
if (WIFEXITED(status)) {
    printf("exit code = %d\n", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("signal = %d\n", WTERMSIG(status));
}
```

其中：

- `WIFEXITED(status)` 判断是否正常结束；
- `WEXITSTATUS(status)` 只在正常结束时提取退出码；
- `WIFSIGNALED(status)` 判断是否被信号终止；
- `WTERMSIG(status)` 提取导致终止的信号编号。

1.3.4 正常等待与异常退出

下面的父进程阻塞等待指定子进程：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    printf("parent: pid = %d\n", getpid());

    pid_t id = fork();
    if (id < 0) {
        perror("fork");
        return 1;
    }

    if (id == 0) {
        int count = 5;
```

```
        while (count--) {
            printf("child: pid = %d, parent = %d\n",
                   getpid(), getppid());
            sleep(1);
        }
        exit(0);
    }

    int status = 0;
    pid_t rid = waitpid(id, &status, 0);
    if (rid == id && WIFEXITED(status)) {
        printf("reaped %d, exit code = %d\n",
               rid, WEXITSTATUS(status));
    }

    return 0;
}
```

运行结果：

```
ZL@VM-0-14-ubuntu:~/code/lesson10$ ./myproc
我是一个父进程：pid: 4095370,ppid: 4082650
我是一个子进程：pid: 4095371,ppid: 4095370
我是一个子进程：pid: 4095371,ppid: 4095370
我是一个子进程：pid: 4095371,ppid: 4095370
我是一个子进程：pid: 4095371,ppid: 4095370
我是一个子进程：pid: 4095371,ppid: 4095370
wait success,退出的子进程是：  4095371,exit_code: 0,exit sig: 0
ZL@VM-0-14-ubuntu:~/code/lesson10$ ps ajx | grep 4082650
4082647 4082650 4082650 4082650 pts/2    4095423 Ss    1002   0:00 -bash
4082650 4095423 4095423 4082650 pts/2    4095423 R+    1002   0:00 ps ajx
4082650 4095424 4095423 4082650 pts/2    4095423 S+    1002   0:00 grep --color=auto 4082650
ZL@VM-0-14-ubuntu:~/code/lesson10$
```

子进程执行期间，父进程阻塞在 `waitpid()` 中；子进程退出后，父进程取得状态并完成回收。

如果把子进程中的等待换成写入空指针：

```
int *p = NULL;
*p = 100;
```

子进程会异常终止。测试结果：

```
ZL@VM-0-14-ubuntu:~/code/lesson10$ ./myproc
我是一个父进程：pid: 4098702,ppid: 4082650
我是一个子进程：pid: 4098703,ppid: 4098702
wait success,退出的子进程是：  4098703,exit_code: 0,exit sig: 11
ZL@VM-0-14-ubuntu:~/code/lesson10$
```

此时退出码没有正常业务含义，真正原因要看终止信号。图中显示子进程被信号 `11` 终止，也就是 `SIGSEGV`。Linux 支持的常见终止信号可以查看：

```
kill -l
```

```
ZL@VM-0-14-ubuntu:~/code/lesson10$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

1.3.5 等待失败与孤儿时间

如果父进程传入的 PID 根本没有对应子进程，`waitpid()` 会立即失败，而不是永久等待。

例如把等待目标误写成 `id + 1`：

```
pid_t rid = waitpid(id + 1, &status, 0);
if (rid < 0) {
    perror("waitpid");
}
```

测试结果：

```
ZL@VM-0-14-ubuntu:~/code/lesson10$ ./myproc
我是一个父进程：pid: 4101849,ppid: 4082650
ret: -1
waitpid: No child processes
我是一个子进程：pid: 4101850,ppid: 1
ZL@VM-0-14-ubuntu:~/code/lesson10$ 我是一个子进程：pid: 4101850,ppid: 1
我是一个子进程：pid: 4101850,ppid: 1
我是一个子进程：pid: 4101850,ppid: 1
我是一个子进程：pid: 4101850,ppid: 1
```

父进程立即收到 `ECHILD: No child processes` 并退出，原先的子进程暂时失去父进程，成为孤儿进程，通常被 PID 为 `1` 的进程收养。图中子进程的父进程 PID 变为 `1`，正对应这一过程。

1.3.6 使用宏提取退出信息

使用标准宏后，不需要在业务代码中反复手写位运算：

```c
if (rid == id && WIFEXITED(status)) {
    printf("exit code = %d\n", WEXITSTATUS(status));
} else if (rid == id && WIFSIGNALED(status)) {
    printf("terminated by signal %d\n", WTERMSIG(status));
}
```

测试结果：

```
ZL@VM-0-14-ubuntu:~/code/lesson10$ ./myproc
我是一个父进程：pid: 4115548,ppid: 4113016
我是一个子进程：pid: 4115549,ppid: 4115548
我是一个子进程：pid: 4115549,ppid: 4115548
我是一个子进程：pid: 4115549,ppid: 4115548
我是一个子进程：pid: 4115549,ppid: 4115548
我是一个子进程：pid: 4115549,ppid: 4115548
wait success,退出的子进程是：  4115549,exit_code: 0
ZL@VM-0-14-ubuntu:~/code/lesson10$
```

宏的可读性和可移植性优于自己复制一份位运算定义。可以把宏背后的位图结构当作理解工具，但正式代码应优先使用系统提供的接口。

1.3.7 阻塞等待、非阻塞轮询与多进程

阻塞等待只做一次等待调用，父进程在此期间不能执行其他任务：

```c
pid_t rid = waitpid(id, &status, 0);
```

非阻塞等待需要父进程轮询：

```c
pid_t rid;
do {
    rid = waitpid(id, &status, WNOHANG);

    if (rid == 0) {
        /* 子进程仍在运行，父进程可以先处理其他任务 */
    }
} while (rid == 0);
```

`WNOHANG` 不会把父进程一直堵在等待函数里，但轮询本身也会消耗 CPU。它是否高效，要看父进程在子进程运行期间是否真的有工作可做。

等待多个子进程时，父进程应先保存所有子进程 PID，再分别回收：

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define CHILD_NUM 5

int main(void)
{
    pid_t children[CHILD_NUM];

    for (int i = 0; i < CHILD_NUM; i++) {
        children[i] = fork();
        if (children[i] == 0) {
            printf("child %d: pid = %d\n", i, getpid());
            sleep(2);
            exit(0);
        } else if (children[i] < 0) {
            perror("fork");
            exit(1);
        }
    }
```

```
    for (int i = 0; i < CHILD_NUM; i++) {
        int status = 0;
        pid_t rid = waitpid(children[i], &status, 0);

        if (rid == children[i] && WIFEXITED(status)) {
            printf("child %d reaped, exit code = %d\n",
                   children[i], WEXITSTATUS(status));
        }
    }

    return 0;
}
```

如果父进程只执行一次 `waitpid(-1, ...)`，只会回收一个已经退出的子进程。创建多个子进程后，应继续回收其余子进程，直到每个 PID 都被处理。

1.4 进程程序替换

1.4.1 `exec` 替换什么，不替换什么

进程程序替换让当前进程停止执行原来的用户空间程序，改为执行磁盘上的另一个程序。

`exec` 成功后：

- 不会创建新进程；
- 当前进程的 PID 通常保持不变；
- `task_struct` 等进程身份和控制信息仍在；
- 用户空间原来的代码、数据和地址空间映射被新程序替换；
- 新的代码、数据、命令行参数和环境变量被装入；
- 原程序 `exec` 调用之后的代码不会继续执行。

`exec` 族函数成功时没有返回值，因为没有地方返回到旧程序；失败时通常返回 `-1`，并设置 `errno`。

加载磁盘上的程序涉及文件访问和进程地址空间重建，属于操作系统负责的能力，因此必须通过系统调用完成。`execve()` 是最靠近内核的入口，其他常用接口是 C 标准库提供的封装。

1.4.2 `exec` 函数族的命名规则

`exec` 函数名后面的字母表示参数组织方式：

- `l`：list，命令参数以可变参数列表传入；
- `v`：vector，命令参数以 `argv` 指针数组传入；
- `p`：path，只给文件名时自动搜索 `PATH`；
- `e`：environment，显式传入新的环境变量数组。

常见函数如下：

函数	参数组织	查找程序	环境变量
`execl()`	列表	需要路径	继承当前环境
`execv()`	数组	需要路径	继承当前环境
`execlp()`	列表	搜索 `PATH`	继承当前环境
`execvp()`	数组	搜索 `PATH`	继承当前环境
`execle()`	列表	需要路径	由 `envp` 明确给出
`execve()`	数组	需要路径	由 `envp` 明确给出
`execvpe()`	数组	搜索 `PATH`	由 `envp` 明确给出，属于 GNU 扩展

POSIX 常见接口包括 `execl`、`execlp`、`execle`、`execv`、`execvp` 和 `execve`。其中 `execve()` 直接对应 Linux 系统调用；`execvpe()` 是额外的 GNU 扩展，不应把它误认为所有系统都必须提供的标准接口。

手册页：

```
exec(3)

NAME
       execl, execlp, execle, execv, execvp, execvpe - execute a file

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       extern char **environ;

       int execl(const char *pathname, const char *arg, ...
                       /*, (char *) NULL */);
       int execlp(const char *file, const char *arg, ...
                       /*, (char *) NULL */);
       int execle(const char *pathname, const char *arg, ...
                       /*, (char *) NULL, char *const envp[] */);
       int execv(const char *pathname, char *const argv[]);
       int execvp(const char *file, char *const argv[]);
       int execvpe(const char *file, char *const argv[], char *const envp[]);
```

参数列表必须以空指针结束。例如：

```
execl("/bin/ls", "ls", "-a", "-l", (char *)NULL);
```

第一个参数通常是程序路径，第二个参数通常作为新程序的 `argv[0]`。`argv[0]` 只是程序约定的名称，不一定与真实路径相同。

1.4.3 `fork()` 与 `exec()` 配合

父进程通常不直接替换自己，而是先创建子进程，再让子进程执行新程序：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t id = fork();
    if (id < 0) {
        perror("fork");
        return 1;
    }
```

```
    if (id == 0) {
        execl("/usr/bin/ls", "ls", "-a", "-l", (char *)NULL);

        /* 只有 exec 失败才会执行到这里 */
        perror("execl");
        _exit(127);
    }

    int status = 0;
    if (waitpid(id, &status, 0) == id && WIFEXITED(status)) {
        printf("ls exit code = %d\n", WEXITSTATUS(status));
    }

    return 0;
}
```

`exec` 调用后紧跟 `exit(0)` 是很危险的写法。如果替换失败，程序会把一个明显的执行错误包装成“成功退出”。更合理的做法是在失败分支打印错误，并返回一个明确的失败码。

带自定义环境变量时，要传递以 `NULL` 结尾的字符串数组：

```
char *const envp[] = {
    "PATH=/bin:/usr/bin",
    "MODE=child",
    NULL
};

char *const argv[] = {
    "env",
    NULL
};

execve("/usr/bin/env", argv, envp);
```

使用 `e` 系列函数时，子进程使用传入的环境变量表，不再自动沿用父进程的那一份。若希望在新环境上继续扩展，应先复制原环境，再加入新变量。

1.4.4 从函数调用理解进程控制

可以把进程级操作与函数调用作一个直观类比：

- `fork()` 创建新的执行路径，类似一次“调用”；
- `exec()` 把这个执行路径换成另一套程序；
- `exit()` 结束执行并交付结果，类似一次“返回”；
- `wait()` 获取结果并回收资源，类似处理返回值和清理现场。

这种对应关系不是语法规定，但很适合理解 shell 为什么能反复运行不同命令。

1.5 用 shell 串起进程控制

1.5.1 shell 的核心循环

shell 本身也是一个进程。它读取一行命令，必要时创建子进程，让子进程通过 `exec()` 执行命令，再等待子进程结束。

核心流程可以写成：

```
显示提示符
    ↓
读取一行输入
    ↓
解析命令和参数
    ↓
是否为内建命令？
    ├─ 是：由 shell 自己执行
    └─ 否：fork 子进程 → execvp 执行程序 → 父进程 waitpid 回收
    ↓
记录退出状态，继续下一轮
```

可以先用下面的代码骨架理解实现思路：

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

int last_status = 0;

int main(void)
```

```
{
    char buffer[1024];
    char *argv[64];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            break;
        }

        buffer[strcspn(buffer, "\n")] = '\0';
        if (buffer[0] == '\0') {
            continue;
        }

        int argc = 0;
        char *token = strtok(buffer, " \t");
        while (token != NULL && argc < 63) {
            argv[argc++] = token;
            token = strtok(NULL, " \t");
        }
        argv[argc] = NULL;

        if (strcmp(argv[0], "exit") == 0) {
            break;
        }

        pid_t id = fork();
        if (id < 0) {
            perror("fork");
            last_status = 1;
            continue;
        }

        if (id == 0) {
            execvp(argv[0], argv);
            perror("execvp");
            _exit(127);
        }

        int status = 0;
        if (waitpid(id, &status, 0) == id) {
            if (WIFEXITED(status)) {
                last_status = WEXITSTATUS(status);
```

```
            } else if (WIFSIGNALED(status)) {
                last_status = 128 + WTERMSIG(status);
            }
        }
    }

    return last_status;
}
```

这段骨架刻意保持简短，重点是展示 `fork`、`exec`、`waitpid` 的配合，而不是完整替代系统 shell。真实 shell 还要处理引号、转义、管道、重定向、作业控制和信号等机制。

1.5.2 为什么有些命令必须由 shell 自己执行

`cd`、`export`、`echo $?` 等命令经常需要直接改变 shell 自身，或者读取 shell 自身保存的状态。

如果 `cd` 放在子进程中执行，改变的只是子进程的工作目录；子进程退出后，父 shell 的工作目录仍然不变。`export` 如果只修改子进程的环境变量，也无法影响父 shell。

因此，这些命令通常作为内建命令，由 shell 自己执行。外部程序则适合交给 `fork + exec`：

```
内建命令：修改 shell 状态 → 必须在 shell 进程内执行
外部命令：创建子进程 → exec 加载新程序 → wait 回收
```

1.5.3 环境变量、命令行参数与退出状态

shell 运行外部程序时，父进程通过 `fork()` 创建子进程，子进程继承当前环境；随后 `execvp()` 或 `execve()` 用新的代码、数据、命令行参数和环境变量表替换子进程。

新程序看到的 `argv` 和 `envp` 就是这样传进去的。父进程和子进程本质上是两个进程，因此：

- 子进程修改自己的环境变量，不会自动修改父 shell；
- shell 执行 `export` 时，必须在 shell 自身中更新环境变量；
- shell 保存最近一次子进程的退出状态，才能支持 `echo $?`；
- 正常结束读取退出码，异常结束记录信号编号。

这样就能把前面几节的知识连接起来：创建、执行、退出、等待和状态回收不是五个孤立接口，而是一条首尾相连的控制链。

1.6 小结

进程控制可以用一条生命周期来理解：

1. `fork()` 创建工作模式相近的新执行流；
2. 父子进程通过不同返回值进入各自分支；
3. 写时拷贝保证数据修改的独立性；
4. 进程通过 `return`、`exit()` 或 `_exit()` 结束，或者被信号异常终止；
5. 父进程通过 `wait()` 或 `waitpid()` 回收子进程并取得退出状态；
6. `exec()` 在保留进程身份的情况下，把用户空间程序替换成另一套代码和数据；
7. shell 用循环反复组织这些操作，从而执行一条又一条命令。

需要始终区分两类结果：

- 子进程正常结束：关注退出码；
- 子进程异常终止：关注终止信号。

并且，只要父进程创建了子进程，就应明确由谁等待、在什么时机等待、如何处理多个子进程。否则就可能留下僵尸进程，或者丢失子进程的退出信息。